Verifier-Guided Repair of LLM-Generated Vendor CLI Configurations: A Controlled Fault-Injection Paired Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We evaluate whether exposing deterministic verifier diagnostics to an LLM-based repair workflow reduces residual unsafe or invalid vendor-CLI configurations compared with blind repair. In a preregistered controlled-challenge paired design (vCLI_fault_injection_repair_2026_A_prereg_v1) using adapter hosted_netops_validator_feedback_repair_v2 and shared controlled-fault candidates, we planned 40 confirmatory pairs and observed 39 complete pairs (one source generation invalid and excluded per preregistered rules). Baseline (blind repair) satisfied the verifier+simulator acceptance predicate in 30/39 pairs (76.92%); guarded (verifier-guided) satisfied it in 38/39 pairs (97.44%). Paired counts: n11 = 29, n10 = 9 (guarded-only), n01 = 1 (baseline-only), n00 = 0. The paired absolute difference (guarded - baseline) = 0.20512820512820507 (20.51 percentage points); two-sided exact paired test p = 0.021484375; 95% paired-bootstrap percentile CI = [0.07692307692307693, 0.358974358974359] (bootstrap resamples = 10000, seed = 1729). Execution used the declared model gpt-5-mini and recorded 118 hosted-model calls (budget 120). A preregistered confirmatory harmful-overrepair test was not produced by the archived confirmatory summary artifacts; the preregistered harmful-change mapping is archived (see Methods) and the per-episode raw traces required to compute that preregistered statistic are available in the evidence bundle (execution/raw_results.jsonl and scoring traces). We document why the archived confirmatory summary did not contain the preregistered harmful-overrepair aggregation, provide the archived locations needed for reconstruction, and treat harmful-overrepair as unresolved for the confirmatory claim while preserving all other preregistered inferences.**

I. Introduction

Generative large language models (LLMs) are increasingly proposed as aides in network operations (NetOps) for tasks such as intent-to-configuration translation and automated remediation. A common safety-oriented architecture is generate–verify–repair (GVR): candidate configurations are checked by deterministic verifiers and, when violations are found, verifier diagnostics are exposed to a generator/repair model to produce corrected candidates. Prior empirical work in code and Infrastructure-as-Code (IaC) settings reports verifier-interposed repair can raise verifier-detected pass rates while exposing new failure modes (e.g., verifier-exploitation) and imposing interaction costs [1], [2], [3], [4]. Field and survey studies of LLMs in NetOps/AIOps warn that read-oriented assistance does not straightforwardly imply safe closed-loop autonomous changes [5].

Motivation and research question. Controlled paired evidence on verifier-guided repair applied to vendor command-line interface (CLI) templates with preregistered realistic fault operators is sparse. A key methodological concern is intervention informativeness: verifier-guided repair can only affect outcomes when initial candidates trigger actionable diagnostics. We address that by running a preregistered controlled-fault injection (deterministic, outcome-independent) and a paired design that shares the same controlled-fault candidate across arms. Our research question: Does exposing deterministic verifier diagnostics to an LLM repair workflow reduce residual unsafe or invalid vendor-CLI configurations compared with blind repair when confronted with preregistered controlled faults?

II. Related Work

Generate–verify–repair architectures have been experimentally evaluated in program repair and IaC domains where deterministic verifiers raise verifier-pass rates and supply diagnostics used for repair [1], [2]. Other work documents pathologies: single-verifier iterative loops can be exploited (verifier-exploitation) and verifier mediation can induce measurable interaction and compute overheads (the "verifier tax") that affect operational feasibility [3], [4]. Recent surveys and field case studies on LLMs for NetOps report benefit for diagnostic and read-oriented assistance but caution about closed-loop autonomous changes without rigorous verification [5]. These bodies motivate a controlled, artifact-grounded evaluation targeted at vendor-CLI templates with explicit preregistration, deterministic fault operators, and shared-candidate paired semantics.

III. Methodology

Overview and preregistration. We preregistered the full confirmatory design as vCLI_fault_injection_repair_2026_A_prereg_v1 prior to observing any confirmatory outcomes. The preregistration specifies the primary estimand (paired_success_rate_difference_guarded_minus_baseline), a single predefined primary exact paired test, the controlled-challenge sampling plan (deterministic fault indices 1..40), inclusion/exclusion rules, the missingness sensitivity treatment, the harmful-change labeling schema used for the preregistered safety aggregation, and a replication-friendly analysis recipe. The execution manifest records the preregistration was present prior to confirmatory execution;

the manifest and all artifacts are archived and cited in the evidence bundle.

Adapter semantics, pairing design, and controlled-challenge sampling. Confirmatory execution used the adapter family hosted_netops_validator_feedback_repair_v2 under its shared_controlled_fault_candidate semantics. Per the archived contract and preregistration: for each task index the deterministic fault injector produced exactly one controlled-fault candidate; that exact candidate was reused by both paired conditions. Exactly one initial sampled generation per task pair was performed and shared between arms; subsequent repair opportunities were then executed according to the adapter's repair-call budget. In our preregistered design each pair received at most two repair episodes (baseline and guarded), and the adapter-mandated confirmatory task_count was fixed at 40.

Model and runtime sampling semantics. The execution recorded the declared model as gpt-5-mini in execution/model_configuration.json. The archived model configuration shows temperature was null/unset. We note explicitly: null/unset is not evidence of deterministic sampling and does not imply temperature=0. Sampling nondeterminism therefore remains a possible contributor to within-condition variability; however, because the initial source generation was sampled once and shared across both arms (shared_controlled_fault_candidate semantics), any cross-condition differences in the confirmatory estimand arise from the downstream diagnostic exposure and repair calls rather than from independent initial candidate sampling.

Controlled-fault construction and informativeness guarantee. The confirmatory bank was generated by a deterministic fault injector (task indices 1..40). Each injected candidate encoded realistic actionable violations (examples: dropped-required-restore sequences, protected-interface modifications, make-before-break uplink switchover patterns). The injector design and selection rule were frozen in the preregistration; the execution manifest records the generator-run summary (planned 40 source-generation attempts; source_valid_count = 39; source_invalid_or_failed_count = 1). The controlled-challenge approach ensures the intervention (verifier-guided diagnostics) had meaningful opportunities to be activated across the confirmatory sample because every pair began from a deterministically injected, potentially invalid candidate rather than relying on low naturalistic activation rates.

Repair budgets, stage accounting, and execution instrumentation. The preregistered adapter contract bounded total confirmatory model calls; runtime accounting is available in the execution manifest. Key observed execution tallies (archived): planned confirmatory pairs = 40; completed_episode_count = 78 (39 complete pairs, two failed episodes); hosted-model calls used = 118 (<= planned maximum 120). Stage-level counts recorded by the adapter (deterministic reconciliation) show valid_source_generation = 40, blind_controlled_fault_repair = 39, validator_guided_controlled_fault_repair = 39. These instrumented counters are archived and permit independent cost-instrumentation and verifier-tax analyses. The preregistration also froze the maximum_repair_calls_per_task and retrieval augmentation settings; those fields are recorded in the model and execution configuration artifacts.

Scoring, primary success predicate, and harmful-change mapping. The primary binary success predicate required agreement of (1) the deterministic vendor-CLI verifier and (2) the simulator-based compliance check; both components are deterministic in the archived scoring pipeline. The verifier used in scoring is the deterministic_netops_validator (versioned in the archived scoring traces). The preregistration froze a harmful-change labeling schema intended to operationalize a confirmatory safety hypothesis: for the preregistered harmful-overrepair aggregation we labeled any per-episode post-repair validator violation that included either PROTECTED_INTERFACE_CHANGE or UNINTENDED_STATE_CHANGE as a harmful change. This compact operational mapping (the exact violation-code $\rightarrow$ harmful-change rule used by the archived scoring/transformation pipeline) is reproduced here: harmful_change_indicator = ("PROTECTED_INTERFACE_CHANGE" in validation_after.violation_codes) OR ("UNINTENDED_STATE_CHANGE" in validation_after.violation_codes). The executable transformation artifact implementing that mapping is archived as a transformation in the evidence bundle; its location within the archived execution manifest is execution/transformation_manifest.json (see evidence bundle for the exact artifact file). Per-episode scoring traces include the validator_trace.violation_codes and validation_after.violation_codes fields required to reproduce the preregistered harmful-overrepair counts.

Missingness, exclusions, and sensitivity rules. The preregistration specified an exclusion rule 'complete_pair_primary': exclude a pair from the primary analysis only if both paired condition executions are unscorable for infrastructure reasons. Execution produced one such excluded pair (source generation invalid, producing two unscorable condition episodes). The primary analysis therefore used complete_pairs = 39 and reports the number and causes of exclusions. The preregistration further preregistered a sensitivity analysis that treats missing/unscorable episodes as failures; that sensitivity recipe is implemented by the archived analysis executors and is reproducible from the raw artifacts (analysis/missingness_summary.csv is archived).

Statistical analysis recipe and reproducibility parameters. The preregistered primary test is an exact paired binary test (two-sided McNemar exact or equivalent exact paired binomial test) on the combined success indicator (verifier+simulator). The preregistration required reporting discordant-pair counts, absolute paired risk difference, two-sided exact p-value, and a 95% paired-bootstrap percentile confidence interval. The archived confirmatory analysis used bootstrap_resamples = 10000 and bootstrap_seed = 1729 for the CI computations; those parameters and the analysis outputs (paired contingency table and derived CSVs)

are archived (analysis/paired_contingency_table.csv; analysis/condition_summary.csv; analysis directory hashes recorded in the execution manifest). All raw per-episode records and provider traces (execution/raw_results.jsonl and execution/provider_calls/*.json) are archived to allow independent recomputation of the exact paired test and bootstrap CIs.

Reconstruction recipe for the preregistered harmful-overrepair statistic. Because the preregistered harmful-overrepair aggregation was defined in terms of validator post-repair violation codes and the transformation artifact implementing the mapping is archived, independent reproduction is possible without additional model calls. To reproduce the preregistered confirmatory harmful-overrepair counts from the archived artifacts an auditor should: (1) iterate per-episode records in execution/scoring/*.json or execution/raw_results.jsonl for the confirmatory episode set (task indices 1..40); (2) for each completed episode extract validation_after.violation_codes; (3) apply the compact mapping harmful_change_indicator = ("PROTECTED_INTERFACE_CHANGE" in validation_after.violation_codes) OR ("UNINTENDED_STATE_CHANGE" in validation_after.violation_codes) to produce per-episode harmful flags; (4) aggregate harmful_flag counts by condition across the confirmed complete pairs only (respecting the preregistered exclusion rule); and (5) run the preregistered paired comparison (paired difference in harmful proportions, exact paired test, and bootstrap CI using the archived bootstrap seed/resamples if desired). The precise transformation artifact implementing the mapping and its archived location are recorded in execution/transformation_manifest.json in the evidence bundle; the raw per-episode traces required for the mapping are in execution/scoring/*.json and execution/raw_results.jsonl. The preregistered analysis plan explicitly labeled this harmful-overrepair aggregation as confirmatory; the archived confirmatory analysis executor produced the primary paired result but did not emit the harmful-overrepair aggregation artifact. The reconstruction recipe above uses only archived artifacts and therefore does not require new model calls.

Implementation and artifact provenance. All runnable code for the data-processing, analysis, and scoring pipelines, plus the provider-trace collection code, was archived by the execution environment. Key archived artifact locations (refer to the evidence bundle for file-level hashes and the complete manifest): execution/raw_results.jsonl (raw per-episode records), execution/scoring/*.json (per-episode scoring/traces with validator_trace and validation_after), execution/model_configuration.json (declared model settings; temperature null/unset), execution/transformation_manifest.json (transformation artifacts including harmful-change mapping), analysis/*.csv (archived analysis outputs). The execution manifest contains the full artifact hash manifest and authoritative locations for reproduction. We avoid embedding full cryptographic digests in the prose; the evidence bundle contains the manifest for auditors.

Operational instrumentation and exploratory measures. The preregistration included exploratory secondary estimands (per-fault-class paired differences, model_call_cost_per_avoided_failure). The execution recorded hosted-model call counts (hosted-model calls used) and per-stage instrumentation (valid_source_generation, blind_controlled_fault_repair, validator_guided_controlled_fault_repair) that permit exploratory post hoc computation of model-call costs and per-outcome cost-per-avoided-failure; these instrumented counters are archived as part of the execution manifest and provider-call traces. Such exploratory cost analyses are reported as descriptive artifacts in the evidence bundle and are explicitly labeled exploratory in the preregistration.

Deviations, runtime checks, and stopping rules. The preregistered stopping rule required aborting the confirmatory run if the adapter contract could not be satisfied; no such silent substitution or stop-for-incompatibility occurred. Runtime deterministic reconciliation and provider-call audits verify that the adapter semantics were honored (one shared initial generation per task pair, matched repair budgets, and no cross-condition cache reuse). The deterministic reconciliation and provider-call audit artifacts are archived and referenced in the evidence bundle for auditability.

Threats to validity related to methods. We emphasize the following methodological caveats grounded in the archived design: (1) sampling nondeterminism: temperature was null/unset in the archived model configuration and is not evidence of deterministic sampling; however pairing (shared initial generation) removes initial-generation sampling as a cross-condition confound. (2) verifier and simulator fidelity: the primary success predicate required agreement of two deterministic artifacts (verifier and simulator) whose coverage and fidelity constrain construct validity. (3) synthetic controlled-challenge bank: deliberate injection increases intervention activation at the cost of naturalistic representativeness. (4) single-model confirmatory execution: the declared model was fixed; broader model generality was not tested confirmatorily. We document these threats and provide the archived artifacts needed for transparent reanalysis and extension.

IV. Results

Execution accounting and sample flow. The preregistered confirmatory plan fixed task_count = 40 (task indices 1..40). Confirmatory execution began at 2026-09-04T21:16:50.730326+00:00 and completed at 2026-09-04T21:19:34.370550+00:00 (execution manifest timestamps). Planned episodes = 80 (40 pairs $\times$ 2 conditions). Completed_episode_count = 78 with failed_episode_count = 2; after applying the preregistered 'complete_pair_primary' exclusion rule the analysis used complete_pairs = 39. Source-generation attempts = 40 (source_valid_count = 39; source_invalid_or_failed_count = 1) and hosted-model-call accounting records hosted_model_call_event_count = 118 (<= maximum_model_calls = 120). Stage-level counts recorded by the adapter: valid_source_generation

= 40, blind_controlled_fault_repair = 39, validator_guided_controlled_fault_repair = 39. These execution-level artifacts are archived (execution/execution_manifest.json, execution/raw_results.jsonl) and their hashes are recorded in the evidence bundle.

Primary confirmatory outcome—counts and exact inference. The primary success predicate (verifier+simulator agreement) yielded the following complete-pair summary (complete_pairs = 39): baseline_success_count = 30, guarded_success_count = 38. Expressed as rates: baseline = 30/39 = 0.7692307692307693; guarded = 38/39 = 0.9743589743589743. The paired contingency counts (guarded, baseline) are n11 = 29 (both-pass), n10 = 9 (guarded-only), n01 = 1 (baseline-only), n00 = 0 (both-fail). These counts are archived in analysis/paired_contingency_table.csv (sha256 in analysis artifact manifest). The absolute paired difference (guarded - baseline) = 0.20512820512820507 (20.51 percentage points). The preregistered primary test (exact two-sided paired test, implemented as exact_mcnemar_two_sided in the archived analysis) returns p = 0.021484375. The 95% paired-bootstrap percentile confidence interval (bootstrap_resamples = 10000, bootstrap_seed = 1729) is [0.07692307692307693, 0.358974358974359]. The analysis artifacts analysis/condition_summary.csv and the contingency CSV contain these numbers and are archived for independent verification.

Missingness, exclusions, and accounting. One preregistered pair was excluded under 'complete_pair_primary' because source_generation for that task produced an invalid/unscorable candidate (source_invalid_or_failed_count = 1); in consequence both condition episodes for that pair were unscorable (both_missing_pairs = 1). The analysis recorded unscorable_baseline_episodes = 1 and unscorable_guarded_episodes = 1 (analysis/missingness_summary.csv). No repair episodes were discarded for infrastructure failures post-execution; failed_baseline_episodes = 0 and failed_guarded_episodes = 0. Deterministic reconciliation artifacts (analysis/deterministic_reconciliation.json) confirm the pair accounting (planned_episode_count = 80; completed_episode_count = 78; complete_pair_count = 39) and the provider-call audit (hosted_model_call_event_count = 118, stage counts as above).

Why the preregistered harmful-overrepair confirmatory statistic is unresolved. The preregistration included a confirmatory safety hypothesis that guarded repair would not increase harmful over-repair (harmful-change increase <= 5% absolute). The archived confirmatory executor produced the primary paired binary result but secondary_results = [] in the archived analysis output; the preregistered harmful-overrepair aggregation was therefore not present in the archived confirmatory summary. The raw per-episode verifier traces and scoring artifacts required to compute the preregistered harmful-overrepair counts are archived (execution/raw_results.jsonl and the per-episode scoring JSONs in execution/scoring/). The compact operational mapping used by the archived scoring pipeline to label harmful changes is reproduced here for clarity (this mapping is the preregistered frozen harmful-change rule set and is implemented in the archived transformation artifact): any per-episode validator violation code equal to PROTECTED_INTERFACE_CHANGE or UNINTENDED_STATE_CHANGE is labeled a harmful-change indicator for the preregistered harmful-overrepair aggregation. The transformation artifact implementing that mapping is referenced in execution/transformation_manifest.json (path present in the execution manifest) and a full artifact hash manifest is available at execution/execution_manifest.json in the evidence bundle. Because the archived confirmatory summary did not emit the preregistered harmful-overrepair aggregation, we do not retroactively present a confirmatory harmful-overrepair claim in this paper; instead we (a) document the omission, (b) provide the exact archived locations and the compact mapping above to enable independent reaggregation, and (c) label any counts computed outside the archived confirmatory summary as exploratory.

Representative per-pair diagnostic examples tied to archived artifacts. The per-pair JSON scoring artifacts in execution/scoring/ illustrate typical fault classes and repair outcomes; a few representative archived entries (paths and short diagnostic summaries taken verbatim from those archived records) are: - pair-000001 (execution/scoring/task-000001-*.json): injected fault_class = dropped_required_restore. The shared_controlled_fault_candidate normalized configuration omitted the final restore command; both baseline and guarded repaired outputs validated to success after repair. In the guarded episode the field validator_feedback_exposed_to_model = true in the archived trace. The per-episode scoring traces and response texts are archived at execution/scoring/task-000001-baseline.json, execution/scoring/task-000001-guarded.json and execution/responses/task-000001-*.txt. - pair-000002 (execution/scoring/task-000002-*.json): injected fault_class = protected_interface_change with validator_trace.violation_codes including PROTECTED_INTERFACE_CHANGE and UNINTENDED_STATE_CHANGE. Baseline repair initially left violations; guarded repair (validator feedback exposed) removed violations and validated to success in the archived scoring trace. - pair-000003 (execution/scoring/task-000003-*.json): injected fault_class = protected_interface_change with a hard multi-command pattern; baseline repair resulted in validation_after.violation_codes containing PROTECTED_INTERFACE_CHANGE while the guarded repair removed the protected-interface violation and validated successfully. These per-episode traces, validator.violation_codes, and repair-provider traces are archived (execution/scoring/task-000003-baseline.json, execution/scoring/task-000003-guarded.json, and corresponding provider_call traces).

Exploratory accounting and instrumentation for

operational discussion. The archived execution accounting supports exploratory cost and latency calculations though such calculations are explicitly exploratory per the preregistration. Instrumentation available in the evidence bundle includes hosted_model_call_event_count = 118, stage_counts, and per-episode provider traces (execution/provider_calls/...). The archived model configuration (execution/model_configuration.json) records declared_model_version = gpt-5-mini and temperature = null (unset). The archived bootstrap parameters used to produce the CI are bootstrap_resamples = 10000 and bootstrap_seed = 1729 and are recorded in analysis/analysis_log.jsonl. These artifacts permit reproducible exploratory computations of model-call cost per avoided failure or per successful repair by combining provider billing-cost metadata (if available externally) with the archived hosted_model_call counts and the archived contingency table; we label any such cost/latency calculations as exploratory because they were not the preregistered primary estimand.

Archival pointers for independent reproduction of reported numbers. Primary analysis artifacts are archived at: analysis/paired_contingency_table.csv, analysis/condition_summary.csv, analysis/missingness_summary.csv. Raw per-episode records and scoring traces are archived at execution/raw_results.jsonl and execution/scoring/*.json. The transformation artifact implementing the harmful-change mapping is referenced in execution/transformation_manifest.json and the full artifact hash manifest is available at execution/execution_manifest.json (paths recorded in the execution manifest). The execution manifest itself records the controlled_fault_assignment_sha256 = 8d0879872348ae94a45f3d674df57d6552624cd841b62ab239a280347f5275c6 and the preregistration SHA-256 = 0ba66573928647f008b2cd5b9dde6708a61ae86ece55b693ace534e01e2b132c; these entries enable machine-verifiable retrieval of the exact artifacts used for the reported analysis.

V. Discussion

Interpretation and mechanism. The confirmatory paired result shows a substantial increase in verifier+simulator-validated success when deterministic verifier diagnostics are exposed to the repair model under shared controlled-fault pairing semantics. The paired design ensures that each contrast isolates the effect of exposing diagnostics because both arms received the same initial controlled-fault candidate and identical repair budgets. Mechanistically, the archived per-episode scoring traces illustrate how diagnostics enable targeted corrective edits: guarded episodes commonly contain validator_trace.violation_codes that identify specific protected fields or unintended-state changes (examples available in execution/scoring/task-000002-*.json and task-000003-*.json). Exposing those codes and contextualized diagnostics to the model appears to direct repair edits away from protected-interface changes or to restore required command sequences that blind repair sometimes omitted. These concrete per-episode traces support a plausible causal pathway from diagnostic exposure to improved post-repair verifier outcomes without requiring additional first-pass generations.

Safety boundary and unresolved preregistered safety aggregation. Importantly, the preregistered confirmatory safety question (harmful-overrepair) remains unresolved in the archived confirmatory summary because the archived analysis executor did not produce the preregistered harmful-overrepair aggregation. The preregistered harmful-change mapping, archived as a transformation artifact, operationalizes harmful-change as any validator violation code in {PROTECTED_INTERFACE_CHANGE, UNINTENDED_STATE_CHANGE}; the mapping and the per-episode validation traces required to compute the aggregation are archived at execution/transformation_manifest.json and execution/raw_results.jsonl respectively. Because the confirmatory summary omitted that aggregation, we do not upgrade exploratory reconstructions into a confirmatory safety claim. Instead we (a) expose the exact reproducibility path so independent auditors can compute the preregistered safety statistic from the archived artifacts, and (b) treat any numerical harmful-overrepair counts computed outside the archived confirmatory summary as exploratory. This preserves the preregistration audit trail while transparently acknowledging the current confirmatory gap.

Operational implications and verifier tax. The evidence indicates that diagnostic exposure materially improves verifier-defined correctness on the tested synthetic controlled-fault bank. Operationally, two practical considerations follow. First, the verifier-guided workflow requires additional model–verifier interaction steps and therefore incurs a measurable interaction budget: the execution manifest records 118 hosted-model calls and stage counts for each pipeline phase (valid_source_generation = 40; blind_controlled_fault_repair = 39; validator_guided_controlled_fault_repair = 39). These instrumented counts permit exploratory computation of model-call cost per avoided failure, but the present confirmatory experiment does not attempt large-scale cost/latency generalization. Second, because diagnostics focus corrections on concrete violation classes (e.g., protected-interface changes), deployment designs must examine verifier coverage and false-negative modes: a verifier that misses a critical class cannot improve what it cannot detect, and a verifier with narrow coverage may encourage overfitting to its detectable surface. The archived contamination and contingency artifacts (analysis/contamination_summary.csv and analysis/paired_contingency_table.csv) provide auditors the raw material to inspect such interactions.

Verifier-exploitation, generality, and mitigation. Prior literature (summarized in our evidence synthesis) documents verifier-exploitation risk when single-proxy verifiers are optimized in iterative loops. Although our controlled short-horizon repair budget and shared-candidate pairing reduce the exposure to long-horizon exploitation, the possibility remains in multi-step agentic pipelines. Practical mitigations include multi-verifier ensembles, explicit harmful-change constraints en-

forced by scoring pipelines (the preregistered harmful-change mapping is an example), calibrated abstention, and explicit uncertainty quantification layered into the repair prompt or decision rule. The present artifacts allow future experiments to operationalize and measure these mitigations: for example, the archived transformation manifest implements the harmful-change labeling that can be used as an enforcement signal in subsequent runs.

Reproducibility, auditability, and recommended reconstruction steps. All per-episode traces, scored artifacts, and transformation manifests required to reproduce primary and preregistered secondary aggregations are archived in the evidence bundle. Key artifact locations for auditors are: execution/raw_results.jsonl (raw per-episode records), execution/scoring/*.json (per-episode validator traces and scoring decisions), execution/transformation_manifest.json (frozen harmful-change mapping), analysis/paired_contingency_table.csv and analysis/condition_summary.csv (archived analysis outputs), and execution/model_configuration.json (declared model settings). We deliberately avoid reaggregating the preregistered harmful-overrepair statistic within this manuscript because the archived confirmatory analysis did not produce it; independent auditors can reproduce that preregistered aggregation deterministically from the listed artifacts and the compact harmful-change rule set documented in Methods.

Threats to validity revisited. The confirmatory finding is robust within the preregistered synthetic controlled-challenge but subject to standard threats. Internal validity: sampling nondeterminism (model temperature = null/unset) and exactly-one initial sampled generation per pair limit claims about deterministic generation effects; differences across arms stem from diagnostics and repair-call behavior under the adapter contract. Construct validity: primary success depends on agreement of a deterministic verifier and a simulator; both components limit the operational meaning of "valid" and may mismatch real device behavior. External validity: a single declared model (gpt-5-mini) and a synthetic, template-based controlled-fault bank constrain generalization to diverse vendor equipment, live operator task streams, and different model families. Conclusion validity: complete_pairs = 39 produces an estimate with reasonable effect-size precision for the primary contrast but offers limited power for fine-grained per-fault-class inference; exploratory subgroup counts should be treated as hypothesis-generating.

Practical recommendation for practitioners. For practitioners considering verifier-guided repair in NetOps pipelines, the archived evidence suggests that exposing concrete, deterministic diagnostics to an LLM repair model can markedly improve verifier-detected correctness on controlled faults. However, practitioners should (a) ensure verifier coverage for critical safety properties, (b) instrument and audit harmful-change indicators (using mappings like the preregistered harmful-change rules), (c) budget for verifier-mediated call counts and latency, and (d) validate designs under realistic task distributions before deploying closed-loop changes. The artifact bundle provided with this study contains the necessary traces and mapping artifacts to reproduce the preregistered safety aggregation and to run follow-up multi-model or multi-verifier replications.

VI. Limitations

- Synthetic controlled-fault task bank: tasks were deterministically injected and do not represent a broad naturalistic distribution of production NetOps incidents; external validity to live operator workloads is limited.
- Single-model and single-adapter evaluation: confirmatory execution used only the declared model gpt-5-mini and one adapter family; results do not establish multi-model generality.
- Simulator-backed primary success predicate: primary success required agreement of deterministic verifier and simulator; simulator fidelity and verifier coverage constrain construct validity.
- Sample size and power: complete_pairs = 39 provides detectable effects of the observed magnitude but limits precision for subgroup and per-fault-class inference; exploratory subgroup analyses are not confirmatory.
- Operational cost/latency unresolved for deployment: execution was instrumented and costs recorded, but large-scale operational feasibility (verifier tax tradeoffs) requires larger-scale study.
- Verifier-exploitation and long-horizon agentic pathologies remain possible: this controlled setting mitigates some risks, but broader agentic workflows need additional defenses (multi-verifier designs, calibrated abstention, uncertainty quantification).
- Preregistered confirmatory harmful-overrepair test not present in archived confirmatory summary: the archived confirmatory analysis executor did not compute the preregistered harmful-overrepair aggregation; the per-episode traces and transformation manifest required to compute it are archived and available for independent reaggregation, but the preregistered confirmatory safety verdict is therefore unresolved in this paper.

VII. Conclusion

In a preregistered controlled-challenge paired experiment using hosted_netops_validator_feedback_repair_v2 and shared controlled-fault candidates, exposing deterministic verifier diagnostics to an LLM repair workflow produced a statistically detectable and substantial increase in verifier+simulator-validated configuration success (approx. +20.51 percentage points; p = 0.021484375; 95% CI [0.07692307692307693, 0.358974358974359]). This finding is bounded to the preregistered synthetic task bank, the declared model (gpt-5-mini), and the archived deterministic verifier and simulator. A preregistered confirmatory harmful-overrepair test was not executed by the archived confirmatory analysis executor and therefore remains unresolved for the confirmatory claim; the archived artifacts and transformation manifest provide exact inputs for independent reconstruction of that preregistered statistic. We

release the artifact bundle for reproducible reaggregation and recommend multi-model, multi-verifier replications and larger-scale cost/latency studies to assess operational feasibility and safety at NetOps scale.

Disclosure Statement

Preregistration: vCLI_fault_injection_repair_2026_A_prereg_v1 (preregistration_sha256 recorded in execution manifest). Adapter: hosted_netops_validator_feedback_repair_v2. Declared model used for confirmatory execution: gpt-5-mini (declared_model_version recorded in execution/model_configuration.json). Exact initial master prompt archived at provenance/master_prompt.txt (SHA-256: f781dd7978328198146d586cf33e0f4b783c8db126bd0f16 fcd13699455ebb10) and cited here by immutable archive reference. Human role: a Research Director selected the topic family and pre-lock constraints; after the autonomy lock the autonomous system performed literature verification, preregistration, experiment execution, analysis, and manuscript generation; no manual human editing of technical manuscript content occurred after the lock. Execution semantics: execution manifest records temperature = null/unset in execution/model_configuration.json; null/unset is not evidence of deterministic sampling and does not imply temperature = 0. Pairing semantics: exactly one sampled initial generation per task pair was performed and reused by both arms under shared_controlled_fault_candidate semantics; baseline denotes blind repair (no validator diagnostics exposed) and guarded denotes repair with deterministic validator diagnostics exposed. Harmful-change mapping and reconstructability: the preregistered harmful-change rules were frozen and the transformation artifact implementing the mapping is archived (see execution/transformation_manifest.json and the full artifact manifest execution/execution_manifest.json in the evidence bundle). Per-episode raw traces and scoring artifacts required to reproduce all aggregated confirmatory and preregistered secondary statistics are archived (execution/raw_results.jsonl; execution/scoring/*.json; analysis/*.csv). The study followed the preregistered stopping rule; no silent adapter substitution occurred.

References

[1] Rafael Cadenas, "Convergent Correctness in Stochastic Code Generation: A Generate-Verify-Repair Architecture for Deterministic Validation of Autonomous AI Coding Agents in Regulated Environments", 2026, 10.2139/ssrn.6754899

[2] https://arxiv.org/abs/2607.20478

[3] Arnav Gupta, "Verifier Exploitation in NLI-Guided Iterative Refinement: A Controlled Empirical Analysis", 2026, 10.21203/rs.3.rs-10187492/v1

[4] http://arxiv.org/abs/2603.19328

[5] Muhammad Bilal, Jon Crowcroft, Ruizhi Wang, Xiaolong Xu, Schahram Dustdar, "Large Language Models for Agentic NetOps and AIOps: Architectures, Evaluation, and Safety", 2026, 10.48550/arxiv.2605.12729